

Виртуальная CAN-плата температурного контроля

Воспроизводимый источник температуры и CAN-сообщений для испытаний без физической платы.

ЗАДАТЬ

Температуру
Пределы диапазона
Запуск / остановку



ОБРАБОТАТЬ

Определить статус
Обнаружить переход
Создать CAN-кадр



НАБЛЮДАТЬ

Статус в vcan0
Смену состояния
Кадры в candump

Пример: temp 850 → 85 °C > 70 °C → overheat → CAN 0x124

Температура задаётся программно. Одна плата, один CAN-интерфейс.

Требование Medium: независимый буфер событий каждого процесса.

Архитектура: от команды до CAN-кадра

ПРОСТРАНСТВО ПОЛЬЗОВАТЕЛЯ

Терминал

printf → /dev/virtual_board

Наблюдатель

candump ← vcan0



ЯДРО LINUX

chardev.c

Копирование и
разбор
Команда → заявка



worker.c + board.c

Порядок выполнения
Состояние под mutex



can.c

CAN_RAW-сокет
kernel_sendmsg

vcan0 создаёт пользователь через ip link; модуль использует готовый интерфейс.

Интерфейсы: команды, события и снимок состояния

УПРАВЛЕНИЕ

write: temp / limits / start / stop
ioctl: SET_LIMITS, SET_PERIOD
boardctl: утилита для ioctl

НАБЛЮДЕНИЕ

read: события из FIFO процесса
GET_STATUS: текущий снимок
sysfs: поля; proc: сводка клиентов

Событие — история; снимок — текущее состояние.

ПРИМЕР: ТЕМПЕРАТУРА УСПЕЛА ВЕРНУТЬСЯ В НОРМУ

В FIFO остаётся событие прежнего перегрева.

GET_STATUS должен вернуть уже normal.

CAN-статус передаёт актуальный снимок.

UAPI: фиксированные поля 32 бита; CAN: явно упакованные поля 16/8 бит.

write / ioctl меняют состояние; read и sysfs показывают разные данные

Температура и передача — независимые состояния

`temp_status` — положение относительно пределов

$T < \text{low}$
`undertemp`

$\text{low} \leq T \leq \text{high}$
`normal`

$T > \text{high}$
`overheat`

`state = stopped`

Температура обновляется
CAN-кадры не отправляются

`state = running`

0x123: статус по таймеру
0x124: смена `temp_status`

`start` / `stop` меняют режим передачи, а не температурный диагноз.
Возврат в диапазон автоматически даёт `normal`; ручного `reset` нет.

`stop` не посылает финальный кадр: `state` во всех отправленных кадрах равен 1.

Температуры: целые десятые доли °C · точные границы входят в норму

Команды и таймер сходятся в одной очереди

write: контекст процесса

Копия команды → work + completion

Отправитель ждёт результат

hrtimer: короткий callback

queue_work(tx_work)

Перенос срока → HRTIMER_RESTART



ОДНА ПОСЛЕДОВАТЕЛЬНАЯ WORKQUEUE

Команда: изменить состояние → при переходе отправить 0x124 → complete

Периодическая работа: снимок → проверка running → отправить 0x123

Согласованный снимок

state_lock защищает копирование.

Отправка выполняется после unlock.

Ограничение очереди

Ожидающая tx_work не добавляется повторно.

Пропущенные периоды не воспроизводятся.

Синхронизация: кто ждёт и какие данные защищены

`command_lock`

Отправитель удерживает до результата команды.

Worker его не берёт: ожидание completion не блокирует исполнителя.

`state_lock`

Защищает изменение состояния и согласованное копирование снимка.

Перед отправкой через сокет блокировка снимается.

`hrtimer callback`

Не берёт mutex и не ждёт workqueue.

Поэтому stop может ждать callback, удерживая state_lock.

Отправитель освобождает заявку после получения результата.

CAN-протокол: восемь байтов состояния

0x123 — периодический статус

0x124 — температурный переход

0–1 · температура	2–3 · нижний	4–5 · верхний	6 · статус	7 · режим
FA 00	64 00	BC 02	00	01
25 °C	10 °C	70 °C	normal	running

BC 02 → 0x02BC → 700 → 70 °C

Числа температуры: signed 16-bit, младший байт первым, шаг 0,1 °C.

ПЕРЕГРЕВ: 85 °C ПРИ ВЕРХНЕМ ПРЕДЕЛЕ 70 °C

124 [8] 52 03 64 00 BC 02 02 01

0x0352 = 850; temp_status = 2; state = 1. Кадр получен в candump.

Персональные FIFO: чтение одного не мешает другому

Согласованная схема: снимок события → отдельная копия каждому процессу.



Полный FIFO теряет новое событие только своего клиента.

Периодические CAN-статусы не занимают место в очередях событий.

До 8 процессов; наследование и предел подключений ещё не испытаны.

Контракт read: ожидание и сохранение остатка

В FIFO ЕСТЬ СОБЫТИЕ

Извлечь запись → сформировать строку
Скопировать доступную часть
Сохранить остаток до следующего read

FIFO ПУСТ

Обычный read: ждать событие
O_NONBLOCK: вернуть EAGAIN
Сигнал: прервать ожидание

Короткий буфер не должен обрезать или потерять событие.

ОДНА СТРОКА — НЕСКОЛЬКО ЧТЕНИЙ

read №1 → начало строки; read №2 → остаток строки
Следующее событие выдаётся после завершения предыдущей строки.

Нулевой read возвращает 0; недоставленный остаток сохраняется при ошибке.

Ожидание читателя не занимает рабочий поток отправки CAN.

Обычный read проверен; короткое чтение, сигналы и ошибки ещё не испытаны

Ошибка init: освобождаем только полученное

Параметры → CAN → workqueue/таймер → устройство → sysfs

ГДЕ ПРОИЗОШЛА ОШИБКА

ЧТО ОСВОБОЖДАЕМ

Проверка параметров

Ничего: ресурсы ещё не выделены

Настройка / bind CAN-сокета

Созданный сокет

Создание workqueue

Ранее созданный CAN-сокет

Регистрация устройства

Частичные регистрации → очередь → сокет

Ошибка sysfs: убрать атрибуты → устройство → класс → cdev → номер.
main.c завершает общие ресурсы по меткам err_worker / err_can.

Завершение: сначала остановить пользователей ресурса

ЗАВИСИМОСТИ РЕСУРСОВ

Callback использует очередь; работа использует сокет.

ВЫГРУЗКА

**Убрать
интерфейсы**

Снять sysfs, затем
удалить устройство



Отменить таймер

hrtimer_cancel
ждёт callback



Завершить работы

destroy_workqueue
ждёт работы



Заккрыть сокет

sock_release
после отправителей

Если освободить очередь раньше таймера, callback может обратиться к освобождённому объекту. Поэтому порядок определяется зависимостями. Ошибка init: откат только успешно полученных ресурсов, в обратном порядке.

Поздние отказы выделения ресурсов ещё не воспроизведены испытаниями.

Штатная выгрузка: открытые файловые операции удерживают модуль через owner

Демонстрация: наблюдаем поведение платы

ПЕРИОД 1000 мс: start → кадры, stop → пауза, start → кадры

2910.447614 → 2911.447622 $\Delta t = 1,000008$ с

2945.447622 → 2967.511155 пауза между кадрами 22,063533 с

Для читаемости показаны последние цифры временных меток. После паузы снова ≈ 1 с.

ТЕМПЕРАТУРНЫЕ УВЕДОМЛЕНИЯ

temp 850 → 0x124, overheat

temp 900 → без нового 0x124

temp 250 → 0x124, normal

stop + temp 850 → без CAN

ПРОВЕРКИ КОНТРАКТА

Обе границы входят в normal.

Неверные temp/limits отклонены.

После stop кадры прекращаются.

Выгрузка завершается успешно.

vsan подтверждает логику и байты, но не физические свойства CAN-шины.

FIFO/ioctl и proc/sys показаны в отдельных сценариях.

Два читателя и единое состояние

ТЕКУЩИЕ ЗНАЧЕНИЯ: `write / ioctl` → одно состояние → `sysfs`

`temp 850` → `GET_STATUS`: `850 / overheat`; `sysfs`: `850`

`limits 100 900`; `period 1000` → `normal / stopped`

`sysfs high_decic = 900, period_ms = 1000` — совпадают с `GET_STATUS`.

ЧИТАТЕЛЬ А — СВОЙ FIFO

`started: 850 / normal`

`temp_transition: 950 / overheat`

`temp_transition: 850 / normal`

`stopped: 850 / normal`

ЧИТАТЕЛЬ Б — СВОЙ FIFO

`started: 850 / normal`

`temp_transition: 950 / overheat`

`temp_transition: 850 / normal`

`stopped: 850 / normal`

Чтение одним процессом не забрало события у второго.

Короткое чтение, переполнение и ошибочные `ioctl` ещё не проверены.

Sysfs: состояние сейчас, read: события

КОМАНДА ЧЕРЕЗ `/dev` → `board.status` → ЧТЕНИЕ `SYSFS`

После загрузки: `state = stopped; temp_status = normal`
`start` → `running; temp 850` → `running / overheat`
`stop` → `stopped / overheat`: передача остановлена, перегрев сохраняется.

`cat /dev/virtual_board`

Новый читатель: пустой FIFO
`read` ждёт новое событие
Событие → строка в терминале
Затем снова ожидание

`cat` атрибута `sysfs`

Чтение вызывает `*_show()`
Поле `board.status` под `mutex`
`sysfs_emit` → текущее значение
`cat` выводит его и завершается

`GET_STATUS` через `ioctl` тоже сразу возвращает текущее состояние.

Начальный снимок в FIFO не добавляется. Это контракт нашего драйвера.

Procs: сводка платы и клиентов

`cat /proc/virtual_board` → СНИМОК ПРИ ОТКРЫТИИ

Плата: `temp=250, low=100, high=700, period_ms=100`

После start: `state=running, temp_status=normal`

`open` → сформировать текст; `read` → части текста → EOF; `release` → освободить.

ЧИТАТЕЛЬ /dev ОТКРЫТ

`clients=1`

`pid=141943 opens=1`

`readers=1 lost_events=0`

Событие `started` доставлено

ЧИТАТЕЛЬ /dev ЗАКРЫТ

`clients=0`

Строки `pid` больше нет

Плата остаётся `running`

Закрытие `cat` не вызывает `stop`

После `stop` и `unload`: пути `/proc`, `/dev` и класс `sysfs` удалены.

Переполнение и чтение `proc` короткими порциями ещё не проверены.

Ошибка CAN: уведомить и продолжить попытки

down → ОШИБКА; **up** → ВОЗОБНОВЛЕНИЕ БЕЗ start

Два отключения → два события `can_error`, `error=-100`

Во время down: `state=running`, `lost_events=0`

Успех сбрасывает прошлую ошибку; повтор одного кода не заполняет FIFO.

ЛОКАЛЬНЫЕ СОБЫТИЯ

started

`can_error`: первый down

`can_error`: второй down

stopped

НАБЛЮДЕНИЕ CAN

До down: кадры раз в секунду

candump: Network is down

После up: кадры возобновились

Повторно запущен только candump

`running` означает разрешение передачи, а не гарантию доставки.

Удаление интерфейса и остальные коды ошибок ещё не проверены.